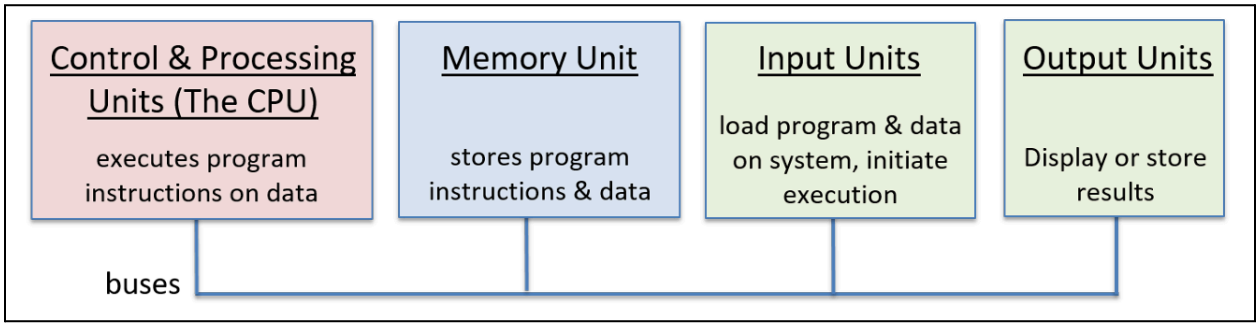
5.5 – Building a Processor: Putting It All Together

The Central Processing Unit (CPU) uses the control and processing units to execute program instructions on data. The Memory Unit stores program instructions & data, the Input Units load the program/data on the system, and the Output Units display results.



A CPU is comprised of:

- The Arithmetic Logic Unit (ALU) for arithmetic (+, -, ×, ÷) and logic (and, or, not…)
- General-purpose Registers for program data storage
- Control circuitry and special-purpose registers for instruction execution
- A Clock that times execution and keeps programs instructed

5.5.1 – The ALU

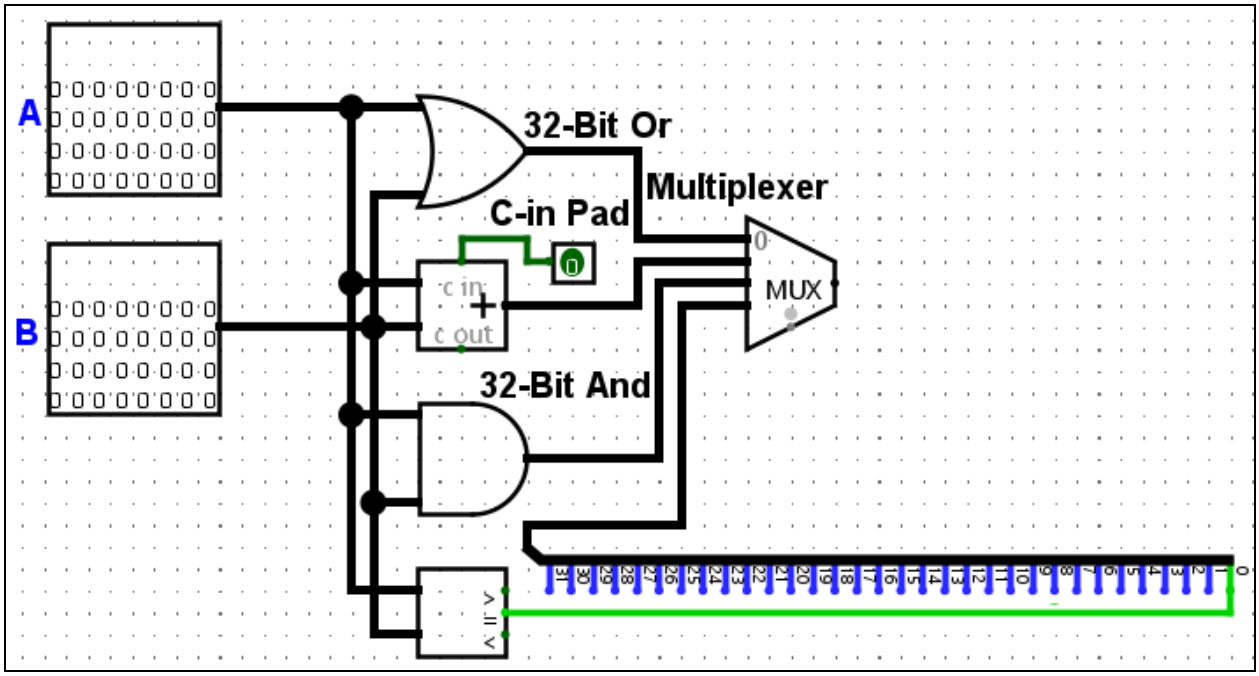
The Arithmetic Logic Unit does arithmetic and logic on *signed (leftmost bit is sign bit)* and *unsigned (no sign bit)* integers. A separate unit is used for floats.

An opcode input specifies the operation type (addition, for example). The ALU outputs the operation result and condition codes that provide info about the result – commonly an indicator for sign, zero, and carry-out. The statement `int x = 6 + 8;` results in a positive, non-zero value with no carry-out. This would be represented with the bit-string 000.

The same ADD instruction can be used to evaluate conditional (if) statements. A program counter (PC), which is used to implement a skip past a conditional statement if the condition fails, contains the address of the next instruction to execute.

A simple ALU sends received inputs to all arithmetic and logic circuits, only outputting what is appropriate using a multiplexer circuit. It takes operands[1] and an opcode as inputs to output the operation's result – this determines the condition code output.

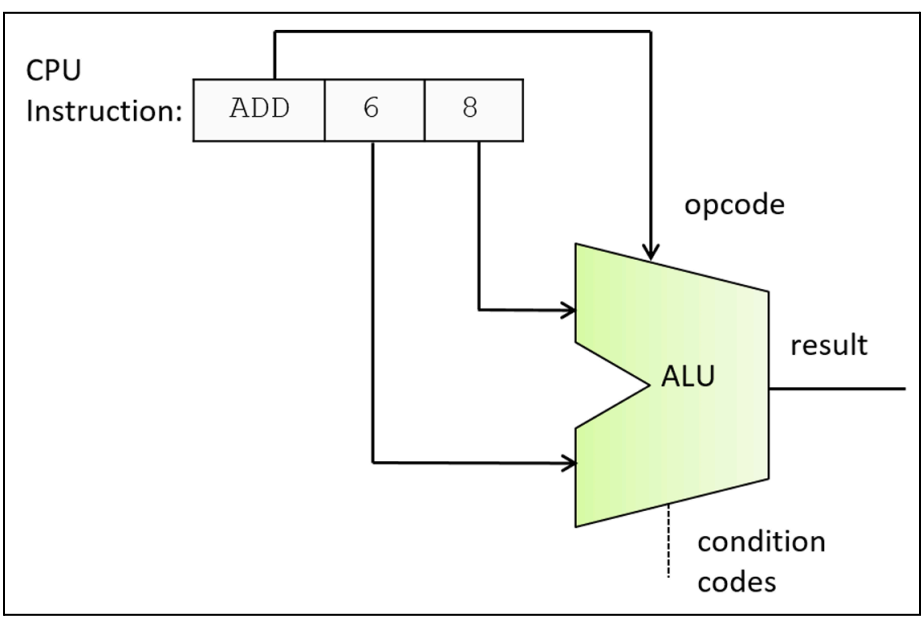
[1] The numbers in an operation.



Logisim representation of Figure 2 from notes, page 2.

An opcode's input is received by the ALU as bits from CPU instructions. For example, ADD could be a bit string with the opcode, the first and second operand, and the result destination. An operand can be represented with a CPU register, its memory address, or its literal value (like 6 or 8).

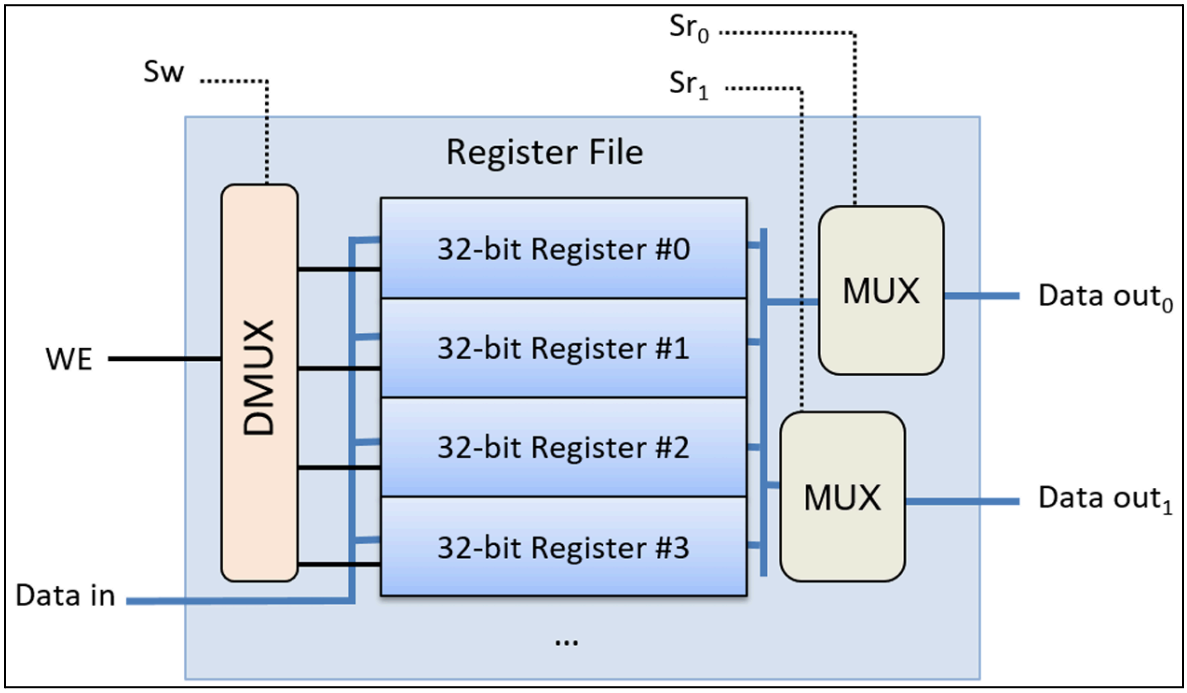
Referencing the ALU above, the opcode requires two bits for one of 4 possible operations. The formula to determine this is $log_2(a) = x$ (or $2^x = a$), where *a* is the number of operations the ALU can execute and *x* is the required amount of bits.



5.5.2 – The Register File

A CPU's general-purpose registers store temporary values, like operands, to be referenced when needed. For example, an ADD instruction adds register 1's value to register 2, then stores the new value in register 3.

General-purpose registers are organized into register file circuits, which consist of register circuits to store values and control circuits to interact with the values. Typically, writing to a register takes one input and simultaneously reading two register values takes two outputs.



Above is a four-register register file circuit. Its outputs are controlled by multiplexers (MUX). Its read inputs (Sr_0 and Sr_1) are fed into the multiplexers to pick the appropriate output.

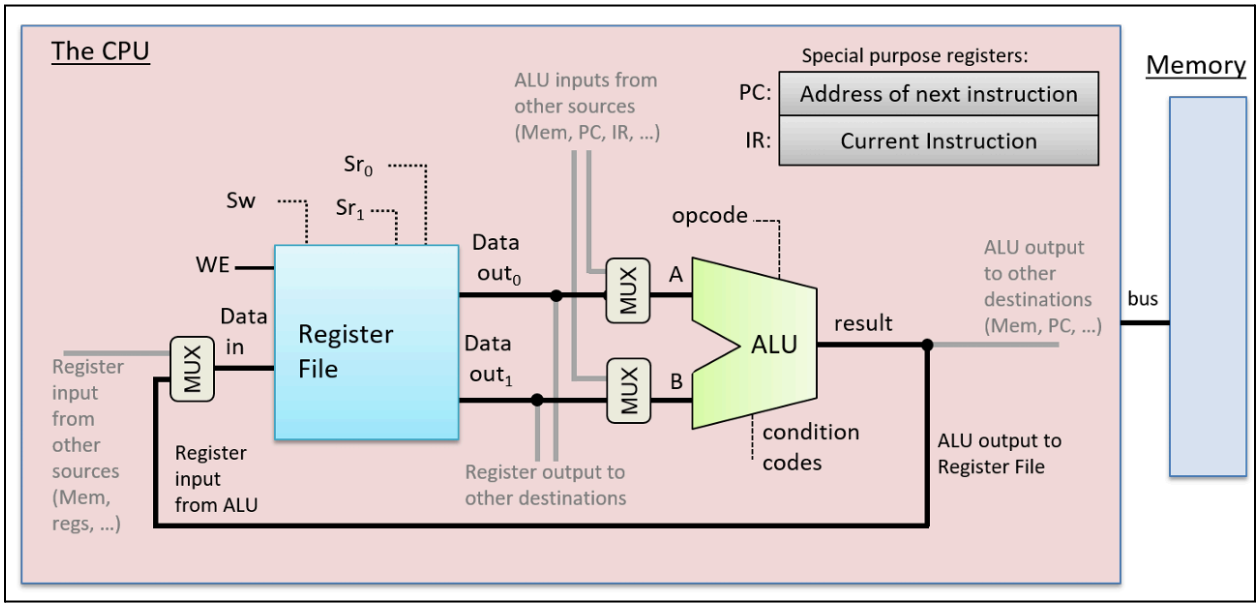
The Data input is sent to every register circuit (32-bit Register). The write enable (WE) input goes through a Demultiplexer (DMUX) circuit, which uses an input to determine which of the register files shouldn't receive a zero. The write selection input (S_w) acts as a demultiplexer switch; only when it is one does the Data input have an effect on the registers.[2]

[2] Compare to a circuit breaker.

Special-Purpose Registers

Special-purpose registers store instruction content and addresses (the program counter is an example). The instruction register (IR) stores the bits of the instruction that the CPU is executing, which are distributed around the CPU as inputs as needed.

5.5.3 – The CPU



The main parts of the CPU are the Register File and the Arithmetic Logic Unit; the former's outputs go in as inputs to the latter and vice versa. Additional circuits within the CPU move data between the two, along with main memory and other PC locations.

The CPU's data path is made of the Register File, ALU, and any circuits connecting them (called buses). The CPU's control path controls the ALU's processes using the Register File's operands. It also sends commands to I/O (input/output) devices and ensures memory allocation for instruction execution goes smoothly. For example, some instructions use direct memory locations instead of general-purpose registers.